EXP NO: 4 Date: 20-8-26	SUPPORT VECTOR MACHINES (SVM)

AIM:

To build an **SVM (Support Vector Machine) model** for binary classification using the Breast Cancer Wisconsin dataset, tune its hyperparameters using GridSearchCV, and evaluate the model using **accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC**.

ALGORITHM:

1. Import the required Python libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn.
2. Load the **Breast Cancer Wisconsin** binary classification dataset from sklearn.datasets.
3. Separate the dataset into input features X and target variable y.
4. Split the dataset into **80% training data and 20% testing data** using train_test_split.
5. Standardize the features using StandardScaler, since SVM is sensitive to feature scaling.
6. Create an SVM classifier using the **RBF kernel**.
7. Define different values of C and gamma for hyperparameter tuning.
8. Use **GridSearchCV with 5-fold cross-validation** to find the best combination of hyperparameters.
9. Train the final SVM model using the best parameters obtained from GridSearchCV.
10. Predict the class labels and probabilities for the test dataset.
11. Calculate **accuracy, precision, recall, F1-score, ROC-AUC**, and the **confusion matrix**.
12. Plot the **ROC curve** and display the model performance.
13. Report the best parameters and evaluation metrics.

CODE:

```
# ========================

# 1) Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

# 2) Load dataset
data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = pd.Series(
    data.target,
    name="target"
)

# 3) Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 4) Standardize features
scaler = StandardScaler()

X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# 5) Define SVM model
```

```python
# 5) Define SVM model
svm = SVC(
    kernel="rbf",
    probability=True,
    random_state=42
)

# 6) Hyperparameter grid
param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": ["scale", 0.01, 0.001, 0.0001]
}

# Grid Search
grid = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    scoring="f1",
    cv=5,
    n_jobs=-1,
    verbose=0
)

grid.fit(X_train_sc, y_train)

print("Best Parameters from Grid Search:")
print(grid.best_params_)

# Get best model
best_svm = grid.best_estimator_

# 7) Train final model
best_svm.fit(X_train_sc, y_train)

# Predictions
y_pred = best_svm.predict(X_test_sc)

# Probability for ROC-AUC
y_prob = best_svm.predict_proba(X_test_sc)[:, 1]

# 8) Evaluation

acc = accuracy_score(y_test, y_pred)

prec = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

rec = recall_score(
```

```python
rec = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

auc = roc_auc_score(
    y_test,
    y_prob
)

cm = confusion_matrix(
    y_test,
    y_pred
)

# Display results
print("\n=== SVM (RBF) – Test Metrics ===")

print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall   : {rec:.4f}")
print(f"F1-Score : {f1:.4f}")
print(f"ROC-AUC  : {auc:.4f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)

# 9) ROC Curve
fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob
)

plt.figure()
```

```python
plt.figure()

plt.plot(
    fpr,
    tpr,
    label=f"SVM (AUC = {auc:.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    color="gray"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve — SVM (RBF)")

plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Best Parameters from Grid Search:
{'C': 10, 'gamma': 0.01}

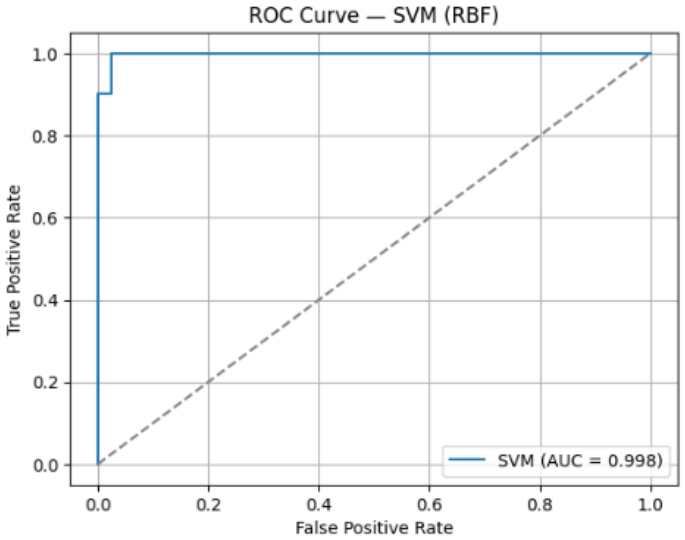
=== SVM (RBF) — Test Metrics ===
Accuracy : 0.9825
Precision: 0.9861
Recall   : 0.9861
F1-Score : 0.9861
ROC-AUC  : 0.9977

Confusion Matrix:
[[41  1]
 [ 1 71]]

Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.98      0.98        42
           1       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```



RESULT:

The SVM model was successfully implemented using the RBF kernel. Hyperparameters were optimized using GridSearchCV, and the model achieved high accuracy with good precision, recall, F1-score, and ROC-AUC. The confusion matrix and ROC curve were also successfully generated.

EXP NO: 4b Date: 20-8-26	ENSEMBLE METHODS: RANDOM FOREST

AIM:

To implement a **Random Forest classifier** for a binary classification task, tune important hyperparameters, evaluate its performance using classification metrics, and identify the most important features.

ALGORITHM:

1. Import the required Python libraries.
2. Load the **Breast Cancer Wisconsin dataset**.
3. Split the dataset into **80% training and 20% testing** data using stratification.
4. Use the raw features since Random Forest does not require feature scaling.
5. Create a RandomForestClassifier.
6. Perform hyperparameter tuning using GridSearchCV.
7. Train the Random Forest model using the best parameters.
8. Predict the test data and calculate **accuracy, precision, recall, F1-score, ROC-AUC**, and confusion matrix.
9. Calculate feature importance and display the **top 10 important features**.
10. Plot the ROC curve and feature importance.

PROGRAM:

```
# 1) Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

# 2) Load dataset
data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = pd.Series(
    data.target,
    name="target"
)
```

```python
# 3) Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 4) Define Random Forest model
rf = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)

# 5) Hyperparameter tuning
param_grid = {
    "n_estimators": [100],
    "max_depth": [None, 10],
    "min_samples_split": [2],
    "min_samples_leaf": [1]
}

grid = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring="f1",
    cv=3,
    n_jobs=-1,
    verbose=0
)

grid.fit(X_train, y_train)

print("Best Parameters (CV):")
print(grid.best_params_)

best_rf = grid.best_estimator_

# 6) Train final model and predict
best_rf.fit(X_train, y_train)

y_pred = best_rf.predict(X_test)
y_prob = best_rf.predict_proba(X_test)[:, 1]

# 7) Evaluation
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob)
cm = confusion_matrix(y_test, y_pred)
```

```python
print("\n=== Random Forest – Test Metrics ===")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall   : {rec:.4f}")
print(f"F1-Score : {f1:.4f}")
print(f"ROC-AUC  : {auc:.4f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)

# 8) Feature Importance - Top 10
importances = pd.Series(
    best_rf.feature_importances_,
    index=X.columns
)

top10 = importances.sort_values(
    ascending=False
).head(10)

print("\nTop 10 Important Features:")
print(top10)

# Plot feature importance
plt.figure(figsize=(10, 6))
top10.sort_values().plot(kind="barh")

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top 10 Feature Importances - Random Forest")
plt.grid(True)
plt.show()

# 9) ROC Curve
fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob
)
```

```python
plt.figure()

plt.plot(
    fpr,
    tpr,
    label=f"Random Forest (AUC = {auc:.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    color="gray"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve – Random Forest")

plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:

```
Best Parameters (CV):
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}

=== Random Forest – Test Metrics ===
Accuracy : 0.9561
Precision: 0.9589
Recall   : 0.9722
F1-Score : 0.9655
ROC-AUC  : 0.9937

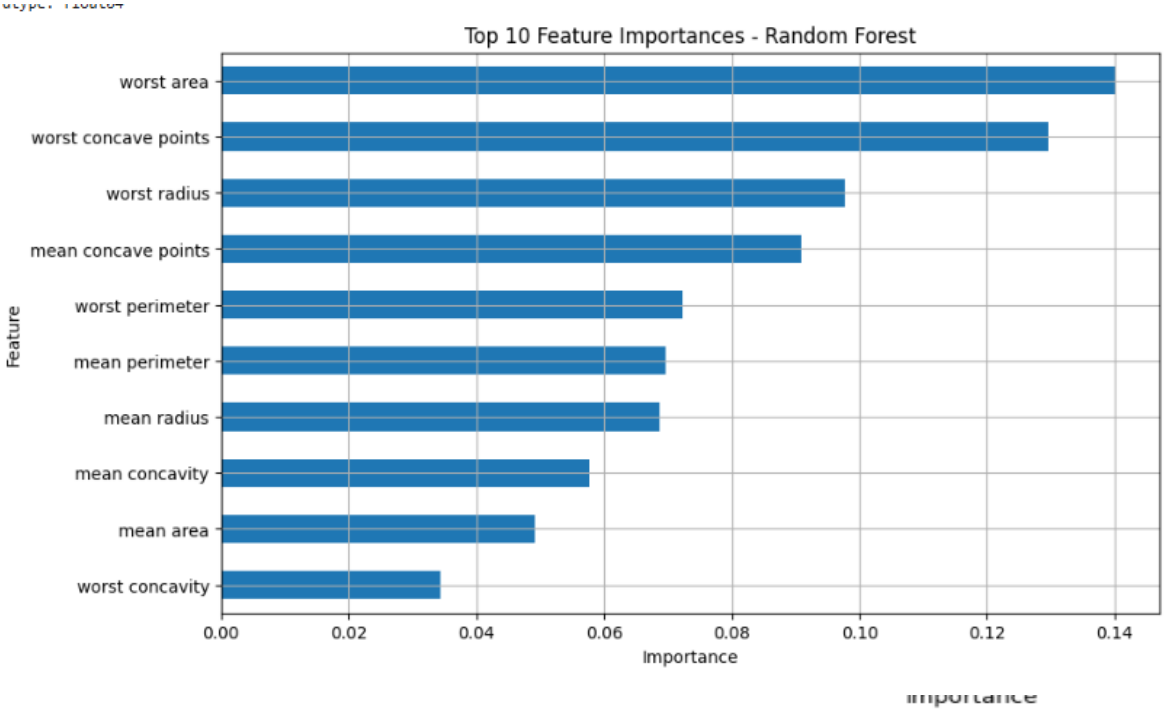
Confusion Matrix:
[[39  3]
 [ 2 70]]

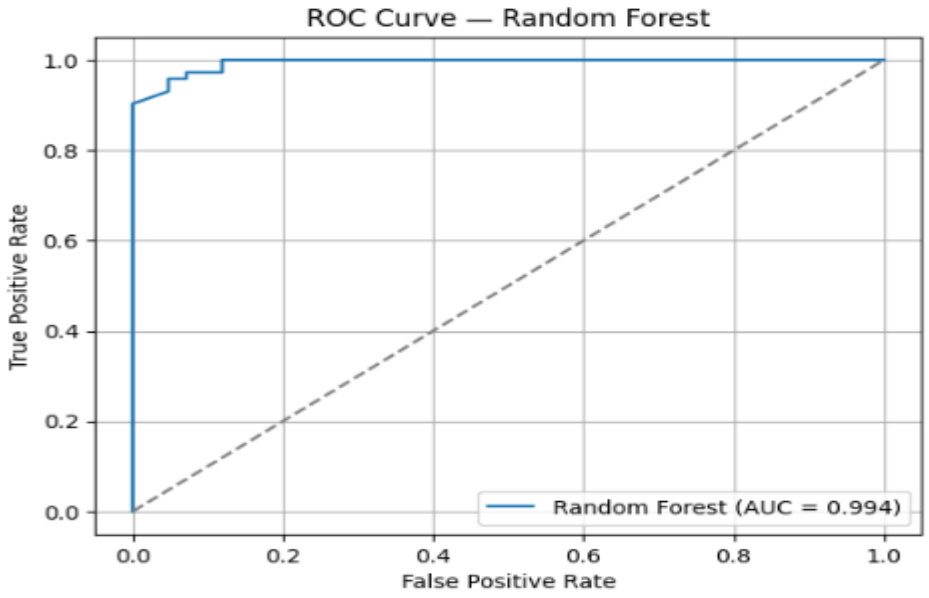
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.93      0.94        42
           1       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114


Top 10 Important Features:
worst area              0.140016
worst concave points    0.129530
worst radius            0.097696
mean concave points     0.090885
worst perimeter         0.072226
mean perimeter          0.069574
mean radius             0.068676
mean concavity          0.057638
mean area               0.049172
worst concavity         0.034340
dtype: float64
```





RESULT:

The **Random Forest classifier** was successfully implemented and optimized using GridSearchCV. The model achieved high classification performance based on accuracy, precision, recall, F1-score, and ROC-AUC. The **confusion matrix, ROC curve, and top 10 important features** were also obtained successfully.